HW5 for CS 6210

You may (and probably should) talk about problems with the each other, with the TA, and with me, providing attribution for any good ideas you might get. Your final write-up should be your own.

Code not executed in *Safe preview*

```
using LinearAlgebra
```

1. Collapsing columns

Suppose $A = QR$ is an economy QR factorization with a dense representation of the (tall thin) Q factor. Now consider the QR factorization of $AP = QRP$, where the action of P is to remove column j of A (or R). Via a sequence of Givens rotations, compute $AP = \tilde{Q}\tilde{R}$ where $\tilde{R}$ is the first $n-1$ rows of $G_1 G_2 \dots G_k(RP)$ (chosen so $\tilde{R}$ is again upper triangular) and $\tilde{Q}$ is the first $n-1$ columns of $QG_k^T G_{k-1}^T \dots G_1^T$. You may use the `LinearAlgebra.givens` functions in Julia to compute the appropriate rotations; you will want to use `lmul!` and `rmul!` to apply the rotations in place efficiently. Your code should take $O(mn)$ time.

We need to compute an updated economy QR factorization after removing column j from A, given $A = QR$ (with Q tall-thin, $mtimesn$, $m \leq n$). The new matrix is AP (where P removes column j), and we compute $AP = \tilde{Q}\tilde{R}$ using Givens rotations in $O(mn)$ time.

Start with RP as R with column j removed ($n \times (n-1)$ matrix). RP is not necessarily upper triangular; removing a middle column introduces subdiagonal elements starting from position (j+1, j) downward along the subdiagonal. To re-triangularize RP, apply Givens rotations from the left to eliminate subdiagonal entries at positions (k+1, k) for k = j to n-1. Each Givens rotation is computed on rows k and k+1 for column k, then applied to the trailing submatrix (rows k:k+1, columns k:n-1) to prevent fill-in propagation. This takes $O(n^2)$ time for R updates (since O(n) rotations, each O(n)). For $Q(mn)$, apply the adjoint (transpose) of each Givens from the right: $Q := QG^T$. Each right-multiplication affects only two columns, costing O(m) per rotation, total O(mn). After updates, $\tilde{R} = RP[1:n-1, 1:n-1]$ (upper triangular, as bottom row is zeroed). $\tilde{Q} = Q[:, 1:n-1]$ (first n-1 columns of the updated Q). The order of Givens (top-down) ensures no additional fill-in, as rotations don't affect lower rows. Computational performance: O(mn) as required, dominated by Q updates. This is more efficient than the naive $qr(QRP)$ which is $O(mn^2)$.

Code not executed in *Safe preview*

```
let
    # Compute QR factorization of the original matrix
    A = rand(10,5)
    F = qr(A)
    Q = F.Q[:,1:5]
    R = F.R

    # Compute QR factorization of the permuted matrix
    j = 2
    AP = A[:,1:5 .!= j]
    Qt, Rt = delcol_qr(Q, R, j)  # Replace with delcol_qr (your code)

    # Check that Qt^T Qt = I, AP = Qt*Rt, and Rt is upper triangular
    norm(Qt'*Qt-I), norm(Qt*Rt-AP)/norm(AP), norm(tril(Rt,-1))
end
```

Code not executed in *Safe preview*

```
function delcol_qr_naive(Q, R, j)
    m, n = size(Q)
    RP = R[:,1:n .!= j]
    F = qr(Q*RP)
    F.Q[:,1:n-1], F.R
end
```

Code not executed in *Safe preview*

```
function delcol_qr(Q, R, j)
    m, n = size(Q)
    RP = hcat(R[:, 1:j-1], R[:, j+1:end])
    for k = j:n-1
        G, r = givens(RP[k, k], RP[k+1, k], k, k+1)
        lmul!(G, RP)
        rmul!(Q, G')
    end
    Qt = Q[:, 1:n-1]
    Rt = RP[1:n-1, 1:n-1]

    Qt, Rt
end
```

2. Regularized residuals

Define the Tikhonov regularized least squares solution operator as

$$F(\lambda) = (A^T A + \lambda^2 I)^{-1} A^T b$$

and let $r(\lambda) = (I - AF(\lambda))b$ be the corresponding residual. Show that

$$\|r(\lambda)\|^2 = \|r(0)\|^2 + \|z(\lambda)\|^2$$

where

$$z(\lambda) = r(\lambda) - r(0) = \lambda^2 F(\lambda)^T x(0).$$

Define $F(\lambda) = (A^T A + \lambda^2 I)^{-1} A^T$, $x(\lambda) = F(\lambda) b$, $r(\lambda) = b - Ax(\lambda) = (I - AF(\lambda))b$. Show that $\|r(\lambda)\|^2 = \|r(0)\|^2 + \|z(\lambda)\|^2$, where $z(\lambda) = r(\lambda) - r(0) = \lambda^2 F(\lambda)^T x(0)$.

Assume A has full column rank for invertibility at $\lambda = 0$. From the definition,
$x(\lambda) = (A^T A + \lambda^2 I)^{-1} A^T b$. Since $A^T b = A^T A x(0)$, we have
$x(\lambda) = (A^T A + \lambda^2 I)^{-1} A^T A x(0)$. Rearrange: $x(\lambda) = x(0) - \lambda^2 (A^T A + \lambda^2 I)^{-1} x(0)$.

Thus, $x(0) - x(\lambda) = \lambda^2 (A^T A + \lambda^2 I)^{-1} x(0)$.

Then, $z(\lambda) = \lambda^2 F(\lambda)^T x(0) = \lambda^2 A (A^T A + \lambda^2 I)^{-1} x(0)$ (since $F(\lambda)^T = A (A^T A + \lambda^2 I)^{-1}$).

Then, $z(\lambda) = A(x(0) - x(\lambda))$.

But $r(\lambda) - r(0) = [b - Ax(\lambda)] - [b - Ax(0)] = A(x(0) - x(\lambda)) = z(\lambda)$.

Thus, $r(\lambda) = r(0) + z(\lambda)$.

Then, $\|r(\lambda)\|^2 = \|r(0) + z(\lambda)\|^2 = \|r(0)\|^2 + 2\langle r(0), z(\lambda) \rangle + \|z(\lambda)\|^2$. To prove, show $\langle r(0), z(\lambda) \rangle = 0$: $r(0) = b - Ax(0)$, so $A^T r(0) = A^T b - A^T A x(0) = 0$ (orthogonality condition for least squares). Thus, $r(0) \perp \mathcal{R}(A)$ (range of A). But $z(\lambda) = A(x(0) - x(\lambda)) \in \mathcal{R}(A)$. Hence, $\langle r(0), z(\lambda) \rangle = 0$.

Therefore, $\|r(\lambda)\|^2 = \|r(0)\|^2 + \|z(\lambda)\|^2$.

This aligns with bias-variance trade-offs and regularization, where $\lambda > 0$ reduces variance but introduces bias, and residuals decompose orthogonally.

3. A nearness problem

For a given matrix $A \in \mathbb{R}^{n \times n}$, write a short code

```
nearest_sum_to_one(A)
```

that computes the closest matrix to A whose entries sum to one. Also provide a tester to sanity check correctness of your code – check that the constraint is satisfied, and that a random directional derivative in a direction consistent with the constraints (i.e. a direction whose entries sum to zero) is zero.

Set problem as $\min_X ||X - A||_F^2$ s.t. $\sum_{i,j} x_{ij} = 1$.

Frobenius norm: $||X - A||_F^2 = \sum_{i,j} (x_{ij} - a_{ij})^2$.

Lagrange: Let $f(X) = \frac{1}{2} \|X - A\|_F^2$, $g(X) = \sum x_{ij} - 1 = 0$.

Gradients: $\nabla f = X - A$, $\nabla g = J$ (all-ones matrix).

Set $X - A = \mu J$, so $X = A + \mu J$. Constraint: $\sum X = \sum A + \mu n^2 = 1$, so $\mu = \frac{1 - \sum A}{n^2}$. Thus, add a constant to each entry of A to adjust the sum to 1. This is the projection onto the affine subspace on nearness problems, where Frobenius is easiest due to differentiability. $O(n^2)$ for sum and broadcast.

Code not executed in *Safe preview*

```
function nearest_sum_to_one(A)
    n = size(A, 1)
    mu = (1 - sum(A)) / (n * n)
    A .+ mu
end
```

Code not executed in *Safe preview*

```
let
    n = 3
    A = rand(n, n)
    X = nearest_sum_to_one(A)
    abs(sum(X) - 1) < 1e-8
    D = rand(n, n)
    D .-= sum(D) / (n * n)
    abs(dot(X - A, D)) < 1e-8
end
```